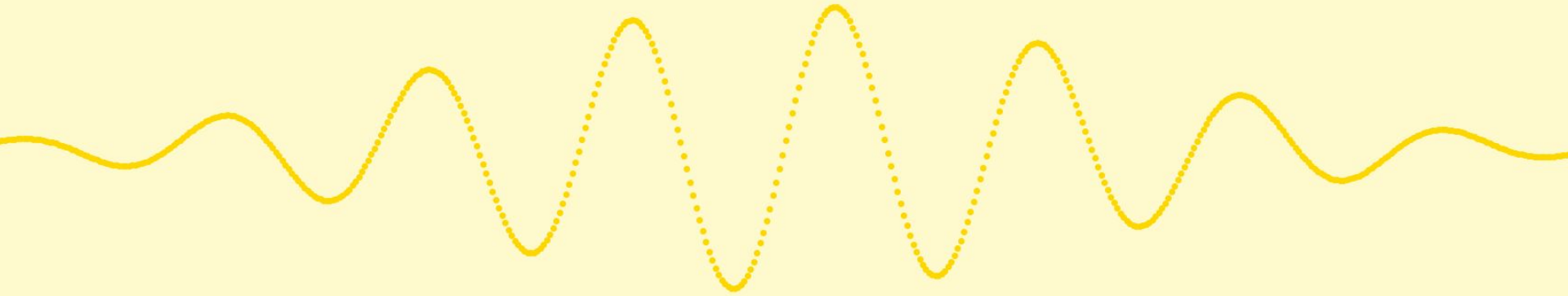


Numerical Methods - 7.1.2026



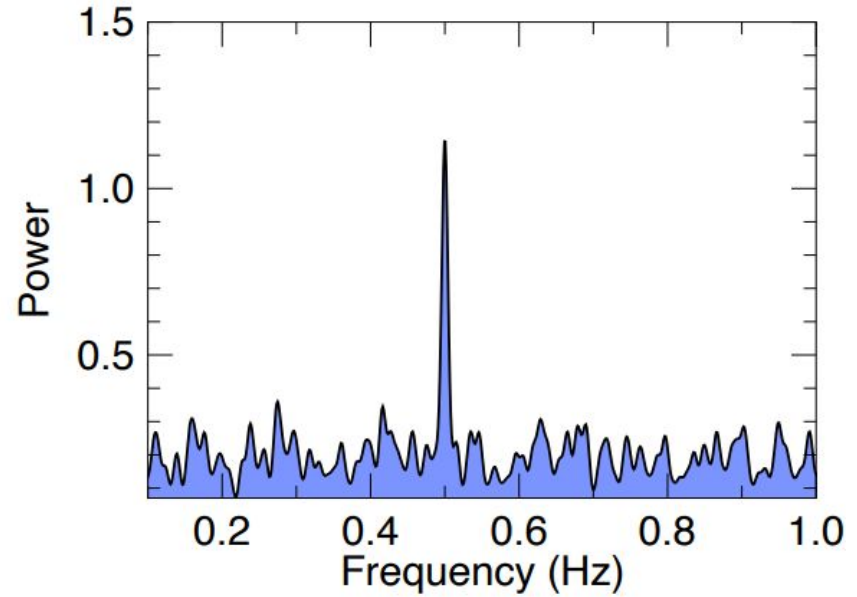
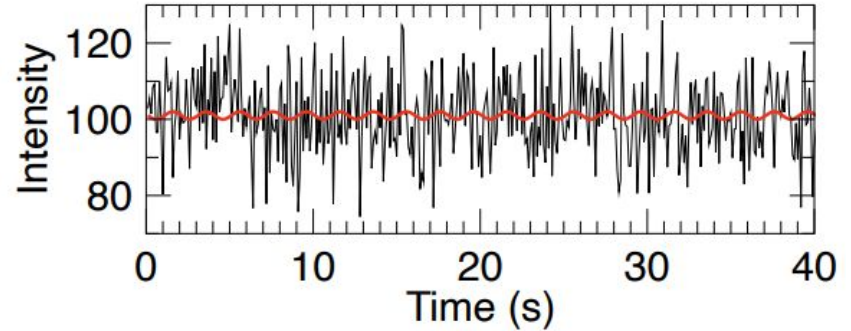
Fast Fourier Transform

Kristina Kislyakova

Introduction

A very large class of important computational problems falls under the general name of “Fourier transform methods” or “spectral methods.”

The Fourier transform is an integral transform which takes a function as an input and produces the related “power spectrum” of frequencies.



Fourier Transform

A physical process can be described either in the **time domain**, by the values of some quantity h as a function of time t , e.g., $h(t)$, or in the **frequency domain**, where the process is specified by giving its amplitude H (generally a complex number indicating phase also) as a function of frequency f , that is $H(f)$, with $-\infty < f < \infty$. For many purposes it is useful to think of $h(t)$ and $H(f)$ as being two different representations of the same function.

Fourier Transform

A physical process can be described either in the **time domain**, by the values of some quantity h as a function of time t , e.g., $h(t)$, or in the **frequency domain**, where the process is specified by giving its amplitude H (generally a complex number indicating phase also) as a function of frequency f , that is $H(f)$, with $-\infty < f < \infty$. For many purposes it is useful to think of $h(t)$ and $H(f)$ as being two different representations of the same function.

We go back and forth between these two representations by means of the Fourier transform equations:

$$H(f) = \int_{-\infty}^{\infty} h(t) e^{2\pi i f t} dt$$
$$h(t) = \int_{-\infty}^{\infty} H(f) e^{-2\pi i f t} df$$

Fourier Transform

A physical process can be described either in the **time domain**, by the values of some quantity h as a function of time t , e.g., $h(t)$, or in the **frequency domain**, where the process is specified by giving its amplitude H (generally a complex number indicating phase also) as a function of frequency f , that is $H(f)$, with $-\infty < f < \infty$. For many purposes it is useful to think of $h(t)$ and $H(f)$ as being two different representations of the same function.

We go back and forth between these two representations by means of the Fourier transform equations:

$$\left. \begin{aligned} H(f) &= \int_{-\infty}^{\infty} h(t) e^{2\pi i f t} dt \\ h(t) &= \int_{-\infty}^{\infty} H(f) e^{-2\pi i f t} df \end{aligned} \right\} \begin{aligned} H(\omega) &= \int_{-\infty}^{\infty} h(t) e^{i\omega t} dt \\ h(t) &= \frac{1}{2\pi} \int_{-\infty}^{\infty} H(\omega) e^{-i\omega t} d\omega \end{aligned}$$

Fourier Transform - Some properties

The Fourier transformation is a **linear operation**. The transform of the sum of two functions is equal to the sum of the transforms. The transform of a constant times a function is that same constant times the transform of the function.

Fourier Transform - Some properties

The Fourier transformation is a **linear operation**. The transform of the sum of two functions is equal to the sum of the transforms. The transform of a constant times a function is that same constant times the transform of the function.

In the time domain, the function $h(t)$ may have one or more special **symmetries**. It might be purely real or purely imaginary or it might be even, $h(t) = h(-t)$, or odd, $h(t) = -h(-t)$. In the frequency domain, these symmetries lead to relationships between $H(f)$ and $H(-f)$:

If . . .	then . . .
$h(t)$ is real	$H(-f) = [H(f)]^*$
$h(t)$ is imaginary	$H(-f) = -[H(f)]^*$
$h(t)$ is even	$H(-f) = H(f)$ [i.e., $H(f)$ is even]
$h(t)$ is odd	$H(-f) = -H(f)$ [i.e., $H(f)$ is odd]

Fourier Transform - Some properties

The Fourier transformation is a **linear operation**. The transform of the sum of two functions is equal to the sum of the transforms. The transform of a constant times a function is that same constant times the transform of the function.

In the time domain, the function $h(t)$ may have one or more special **symmetries**. It might be purely real or purely imaginary or it might be even, $h(t) = h(-t)$, or odd, $h(t) = -h(-t)$. In the frequency domain, these symmetries lead to relationships between $H(f)$ and $H(-f)$:

If . . .	then . . .
$h(t)$ is real	$H(-f) = [H(f)]^*$
$h(t)$ is imaginary	$H(-f) = -[H(f)]^*$
$h(t)$ is even	$H(-f) = H(f)$ [i.e., $H(f)$ is even]
$h(t)$ is odd	$H(-f) = -H(f)$ [i.e., $H(f)$ is odd]

These symmetries can be used to increase computational efficiency.

Fourier Transform - Some properties

If $h(t) \iff H(f)$ is a transform pair, then other transform pairs are:

Fourier Transform - Some properties

If $h(t) \Longleftrightarrow H(f)$ is a transform pair, then other transform pairs are:

$$h(at) \Longleftrightarrow \frac{1}{|a|} H\left(\frac{f}{a}\right) \quad \text{“time scaling”}$$

$$\frac{1}{|b|} h\left(\frac{t}{b}\right) \Longleftrightarrow H(bf) \quad \text{“frequency scaling”}$$

$$h(t - t_0) \Longleftrightarrow H(f) e^{2\pi i f t_0} \quad \text{“time shifting”}$$

$$h(t) e^{-2\pi i f_0 t} \Longleftrightarrow H(f - f_0) \quad \text{“frequency shifting”}$$

Fourier Transform - Some properties

With two functions $h(t)$ and $g(t)$, and their corresponding Fourier transforms $H(f)$ and $G(f)$, we can form two combinations of special interest.

Fourier Transform - Some properties

With two functions $h(t)$ and $g(t)$, and their corresponding Fourier transforms $H(f)$ and $G(f)$, we can form two combinations of special interest.

Convolution Theorem – The Fourier transform of the convolution is the product of the individual Fourier transforms:

$$g * h \equiv \int_{-\infty}^{\infty} g(\tau)h(t - \tau) d\tau \quad \longrightarrow \quad g * h \Longleftrightarrow G(f)H(f)$$

Fourier Transform - Some properties

With two functions $h(t)$ and $g(t)$, and their corresponding Fourier transforms $H(f)$ and $G(f)$, we can form two combinations of special interest.

Convolution Theorem – The Fourier transform of the convolution is the product of the individual Fourier transforms:

$$g * h \equiv \int_{-\infty}^{\infty} g(\tau)h(t - \tau) d\tau \longrightarrow g * h \Longleftrightarrow G(f)H(f)$$

Correlation Theorem – Multiplying the Fourier transform of one function by the complex conjugate of the Fourier transform of the other gives the Fourier transform of their correlation:

$$\text{Corr}(g, h) \equiv \int_{-\infty}^{\infty} g(\tau + t)h(\tau) d\tau \longrightarrow \text{Corr}(g, h) \Longleftrightarrow G(f)H^*(f)$$

The correlation of a function with itself is called its autocorrelation.

Fourier Transform - Power Spectral Density

The **total power** in a signal is the same whether we compute it in the time domain or in the frequency domain:

$$\text{Total Power} \equiv \int_{-\infty}^{\infty} |h(t)|^2 dt = \int_{-\infty}^{\infty} |H(f)|^2 df$$

If we want to know how much power is contained in the frequency interval between f and $f + df$, we do not usually distinguish between positive and negative f , but consider f as varying from 0 to $+\infty$. In such cases, the **one-sided power spectral density (PSD)** of the function h is defined as:

$$P_h(f) \equiv |H(f)|^2 + |H(-f)|^2 \quad 0 \leq f < \infty$$

If $h(t)$ goes endlessly from $-\infty < t < \infty$, then its total power and PSD will, in general, be infinite. Of interest then is the power spectral density per unit time. This is computed by taking a long, but finite, stretch of the function $h(t)$, computing its PSD, and then dividing the resulting PSD by the length of the stretch used.

Fourier Transform - Discrete data

In computational work, especially with experimental data, we are almost never given a continuous function $h(t)$ to work with, but are given a list of measurements of h for a discrete set of times. Let Δ denote the time interval between consecutive samples, so that the sequence of sampled values is

$$h_n = h(n\Delta) \quad n = \dots, -3, -2, -1, 0, 1, 2, 3, \dots$$

The reciprocal of Δ is called **the sampling rate**; if Δ is measured in seconds, for example, then the sampling rate is the number of samples recorded per second.

Fourier Transform - Discrete data

In computational work, especially with experimental data, we are almost never given a continuous function $h(t)$ to work with, but are given a list of measurements of h for a discrete set of times. Let Δ denote the time interval between consecutive samples, so that the sequence of sampled values is

$$h_n = h(n\Delta) \quad n = \dots, -3, -2, -1, 0, 1, 2, 3, \dots$$

The reciprocal of Δ is called **the sampling rate**; if Δ is measured in seconds, for example, then the sampling rate is the number of samples recorded per second.

For any sampling interval Δ , there is also a special frequency f_c , called the **Nyquist critical frequency**, given by:

$$f_c \equiv \frac{1}{2\Delta}$$



Critical sampling of a sine wave is 2 sample points per cycle. If we measure time in units of the sampling interval Δ , the Nyquist critical frequency is just the constant $1/2$.

Nyquist critical frequency

The Nyquist critical frequency is important for 2 reasons:

Nyquist critical frequency

The Nyquist critical frequency is important for 2 reasons:

- **sampling theorem** – If a continuous function $h(t)$, sampled at an interval Δ , happens to be bandwidth limited to frequencies smaller in magnitude than f_c , i.e., if $H(f) = 0$ for all $|f| \geq f_c$, then $h(t)$ is completely determined by its samples h_n .

$$h(t) = \Delta \sum_{n=-\infty}^{+\infty} h_n \frac{\sin[2\pi f_c(t - n\Delta)]}{\pi(t - n\Delta)}$$

Nyquist critical frequency

The Nyquist critical frequency is important for 2 reasons:

- **sampling theorem** – If a continuous function $h(t)$, sampled at an interval Δ , happens to be bandwidth limited to frequencies smaller in magnitude than f_c , i.e., if $H(f) = 0$ for all $|f| \geq f_c$, then $h(t)$ is completely determined by its samples h_n .

$$h(t) = \Delta \sum_{n=-\infty}^{+\infty} h_n \frac{\sin[2\pi f_c(t - n\Delta)]}{\pi(t - n\Delta)}$$

- **aliasing** – Effect of sampling a continuous function that is not bandwidth limited to $< f_c$. Any frequency component outside of the frequency range $(-f_c, f_c)$ is *aliased* (falsely translated) into that range by discrete sampling. If a continuous function has been competently sampled, when we estimate its Fourier transform from the discrete samples, we can assume that its Fourier transform is equal to zero outside of the range $(-f_c, f_c)$. Then we check whether indeed the Fourier transform is approaching zero as the frequency approaches $-f_c$ and f_c .

Discrete Fourier Transform

We now estimate the Fourier transform of a function from a finite number of its sampled points. Suppose that we have N consecutive sampled values

$$h_k \equiv h(t_k), \quad t_k \equiv k\Delta, \quad k = 0, 1, 2, \dots, N - 1$$

so that the sampling interval is Δ , and N is even. If $h(t)$ is nonzero only in a finite interval of time, then we consider that whole interval of time to be contained in the range of the N points given; otherwise, we consider that the sampled points are “typical” of what $h(t)$ looks like at all other times.

Discrete Fourier Transform

We now estimate the Fourier transform of a function from a finite number of its sampled points. Suppose that we have N consecutive sampled values

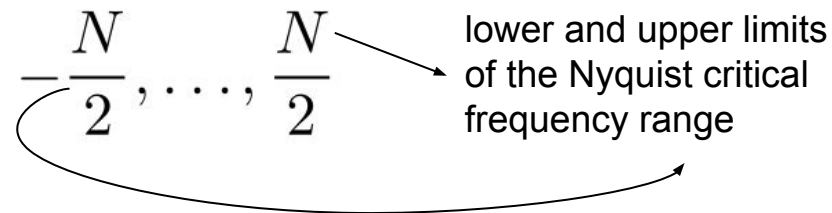
$$h_k \equiv h(t_k), \quad t_k \equiv k\Delta, \quad k = 0, 1, 2, \dots, N - 1$$

so that the sampling interval is Δ , and N is even. If $h(t)$ is nonzero only in a finite interval of time, then we consider that whole interval of time to be contained in the range of the N points given; otherwise, we consider that the sampled points are “typical” of what $h(t)$ looks like at all other times.

Instead of trying to estimate the Fourier transform $H(f)$ at all values of f in the range $(-f_c, f_c)$, let's estimate it only at the discrete values

$$f_n \equiv \frac{n}{N\Delta}, \quad n = -\frac{N}{2}, \dots, \frac{N}{2}$$

lower and upper limits
of the Nyquist critical
frequency range



Discrete Fourier Transform

Now we approximate the integral in the definition of $H(f)$ with a sum:

$$H(f_n) = \int_{-\infty}^{\infty} h(t) e^{2\pi i f_n t} dt \approx \sum_{k=0}^{N-1} h_k e^{2\pi i f_n t_k} \Delta = \Delta \sum_{k=0}^{N-1} h_k e^{2\pi i k n / N}$$

Discrete Fourier Transform

Now we approximate the integral in the definition of $H(f)$ with a sum:

$$H(f_n) = \int_{-\infty}^{\infty} h(t) e^{2\pi i f_n t} dt \approx \sum_{k=0}^{N-1} h_k e^{2\pi i f_n t_k} \Delta = \Delta \sum_{k=0}^{N-1} h_k e^{2\pi i k n / N}$$

$= H_n$, **discrete Fourier transform** of the N points h_k



Discrete Fourier Transform

Now we approximate the integral in the definition of $H(f)$ with a sum:

$$H(f_n) = \int_{-\infty}^{\infty} h(t) e^{2\pi i f_n t} dt \approx \sum_{k=0}^{N-1} h_k e^{2\pi i f_n t_k} \Delta = \Delta \sum_{k=0}^{N-1} h_k e^{2\pi i k n / N}$$

$= H_n$, **discrete Fourier transform** of the N points h_k



The discrete Fourier transform maps N complex numbers (h_k) into N complex numbers (H_n). It does not depend on any dimensional parameter, such as the time scale Δ . It is periodic in n , with period N , therefore: $H_{-n} = H_{N-n}$ ($n = 1, 2, \dots$): zero frequency corresponds to $n = 0$, positive frequencies $0 < f < f_c$ correspond to $1 \leq n \leq N/2 - 1$, while negative frequencies $-f_c < f < 0$ correspond to $N/2 + 1 \leq n \leq N - 1$. The value $n = N/2$ corresponds to both $f = f_c$ and $f = -f_c$.

Discrete Fourier Transform

Now we approximate the integral in the definition of $H(f)$ with a sum:

$$H(f_n) = \int_{-\infty}^{\infty} h(t) e^{2\pi i f_n t} dt \approx \sum_{k=0}^{N-1} h_k e^{2\pi i f_n t_k} \Delta = \Delta \sum_{k=0}^{N-1} h_k e^{2\pi i k n / N}$$



$= H_n$, **discrete Fourier transform** of the N points h_k

The discrete Fourier transform maps N complex numbers (h_k) into N complex numbers (H_n). It does not depend on any dimensional parameter, such as the time scale Δ . It is periodic in n , with period N , therefore: $H_{-n} = H_{N-n}$ ($n = 1, 2, \dots$): zero frequency corresponds to $n = 0$, positive frequencies $0 < f < f_c$ correspond to $1 \leq n \leq N/2 - 1$, while negative frequencies $-f_c < f < 0$ correspond to $N/2 + 1 \leq n \leq N - 1$. The value $n = N/2$ corresponds to both $f = f_c$ and $f = -f_c$.

Discrete inverse Fourier transform:
$$h_k = \frac{1}{N} \sum_{n=0}^{N-1} H_n e^{-2\pi i k n / N}$$

Fast Fourier Transform

How can we compute the discrete Fourier transform of N points?

$$W \equiv e^{2\pi i/N}$$

complex number

$$H_n = \sum_{k=0}^{N-1} W^{nk} h_k$$

discrete Fourier transform

The vector of h_k is multiplied by a matrix whose (n, k) -th element is the constant W to the power $n \times k$. The matrix multiplication produces a vector with components H_n and it requires N^2 complex multiplications, so it appears that the discrete Fourier transform is of order $O(N^2)$.

Fast Fourier Transform

How can we compute the discrete Fourier transform of N points?

$$W \equiv e^{2\pi i/N} \quad \text{complex number}$$

$$H_n = \sum_{k=0}^{N-1} W^{nk} h_k \quad \text{discrete Fourier transform}$$

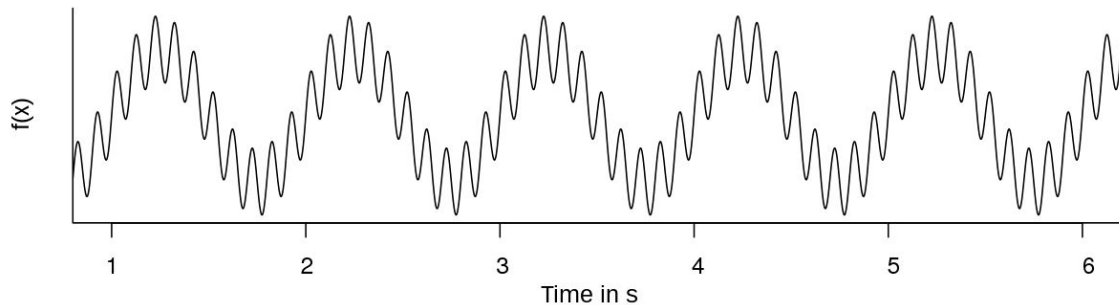
The vector of h_k is multiplied by a matrix whose (n, k) -th element is the constant W to the power $n \times k$. The matrix multiplication produces a vector with components H_n and it requires N^2 complex multiplications, so it appears that the discrete Fourier transform is of order $O(N^2)$.

BUT the discrete Fourier transform can be computed in $O(N \log_2 N)$ operations with an algorithm called the **fast Fourier transform**, or **FFT**.

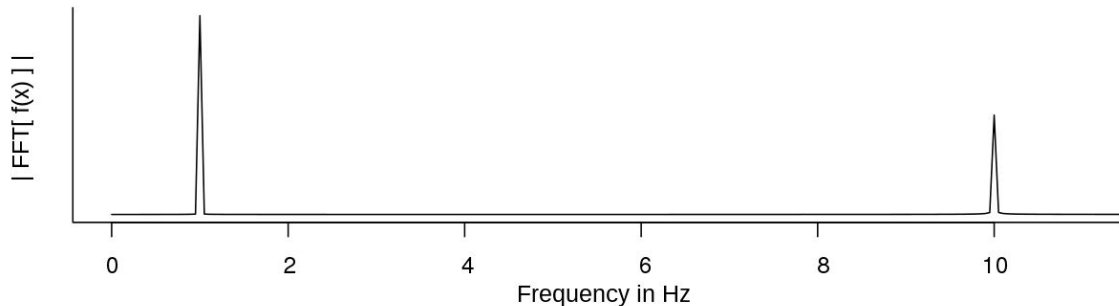
Fast Fourier Transform

The existence of a **Fast Fourier Transform algorithm** became generally known only in the mid-1960s, from the work of Cooley and Tukey. Retrospectively, we now know that efficient methods for computing the discrete Fourier transform had been independently discovered, and in some cases implemented, by as many as a dozen individuals, starting with Gauss in 1805!

$$\text{signal } f(x) = 2 \sin x + \sin 10x$$



Signal that was fourier-transformed via FFT (frequency spectrum)



Fast Fourier Transform

Danielson and Lanczos in 1942 provided one of the clearest derivations of the algorithm. They showed that a discrete Fourier transform of length N can be rewritten as the sum of two discrete Fourier transforms, each of length $N/2$. One of the two is formed from the even-numbered points of the original N , the other from the odd-numbered points. This is the proof:

$$\begin{aligned} F_k &= \sum_{j=0}^{N-1} e^{2\pi i j k / N} f_j \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k (2j) / N} f_{2j} + \sum_{j=0}^{N/2-1} e^{2\pi i k (2j+1) / N} f_{2j+1} \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j} + W^k \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j+1} \\ &= F_k^e + W^k F_k^o \end{aligned}$$

Fast Fourier Transform

Danielson and Lanczos in 1942 provided one of the clearest derivations of the algorithm. They showed that a discrete Fourier transform of length N can be rewritten as the sum of two discrete Fourier transforms, each of length $N/2$. One of the two is formed from the even-numbered points of the original N , the other from the odd-numbered points. This is the proof:

$$\begin{aligned} F_k &= \sum_{j=0}^{N-1} e^{2\pi i j k / N} f_j \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k (2j) / N} f_{2j} + \sum_{j=0}^{N/2-1} e^{2\pi i k (2j+1) / N} f_{2j+1} \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j} + W^k \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j+1} \\ &= F_k^e + W^k F_k^o \end{aligned}$$

k -th ($k = 0, \dots, N$)
components of the Fourier
transform of length $N/2$
formed from the even/odd
components of the original f_j

Fast Fourier Transform

Danielson and Lanczos in 1942 provided one of the clearest derivations of the algorithm. They showed that a discrete Fourier transform of length N can be rewritten as the sum of two discrete Fourier transforms, each of length $N/2$. One of the two is formed from the even-numbered points of the original N , the other from the odd-numbered points. This is the proof:

$$\begin{aligned} F_k &= \sum_{j=0}^{N-1} e^{2\pi i j k / N} f_j \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k (2j) / N} f_{2j} + \sum_{j=0}^{N/2-1} e^{2\pi i k (2j+1) / N} f_{2j+1} \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j} + W^k \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j+1} \\ &= F_k^e + W^k F_k^o \end{aligned}$$

it can be used
recursively!!!

k -th ($k = 0, \dots, N$)
components of the Fourier
transform of length $N/2$
formed from the even/odd
components of the original f_j

Fast Fourier Transform

Recursion – Having reduced the problem of computing F_k to that of computing F_k^e and F_k^o , we can do the same reduction of F_k^e to the problem of computing the transform of its $N/4$ even-numbered and $N/4$ odd-numbered data, F_k^{ee} and F_k^{eo} . The easiest case is the one in which the original N is an integer power of 2. With this restriction on N , we can continue applying the **Danielson-Lanczos Lemma** until we have subdivided the data all the way down to transforms of length 1. What is the Fourier transform of length one? For every pattern of $\log_2 N$ e's and o's, there is a one-point transform that is just one of the input numbers f_n :

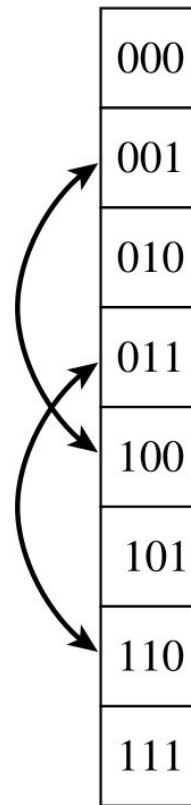
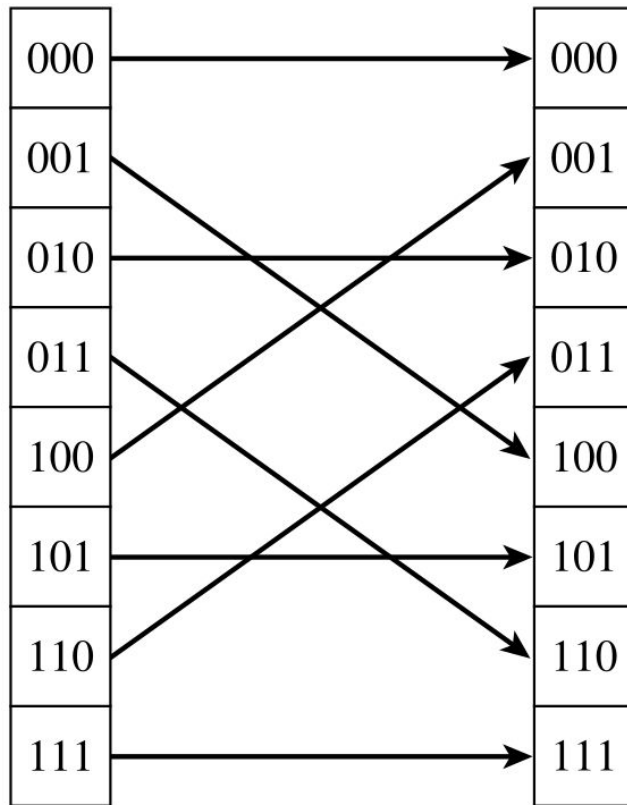
$$F_k^{eooeooeo\cdots oee} = f_n \quad \text{for some } n$$

Now we need to figure out which value of n corresponds to which pattern of e's and o's here. The answer is: Reverse the pattern of e's and o's, then let $e = 0$ and $o = 1$, and you will have, in binary the value of n .

Fast Fourier Transform

Bit reversal

between
2 arrays

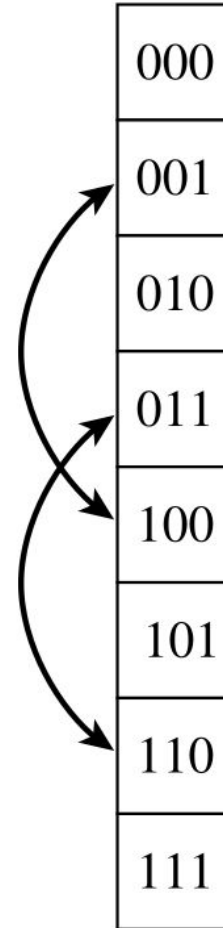


in place

Fast Fourier Transform

Combination – So, we take the original vector of data f_j and rearrange it into bit-reversed order, so that the individual numbers are in the order not of j , but of the number obtained by bit-reversing j .

Then the application of the recursive Danielson-Lanczos Lemma becomes simple. The points as given are the one-point transforms. We combine adjacent pairs to get two-point transforms, then combine adjacent pairs of pairs to get 4-point transforms, and so on, until the first and second halves of the whole data set are combined into the final transform.

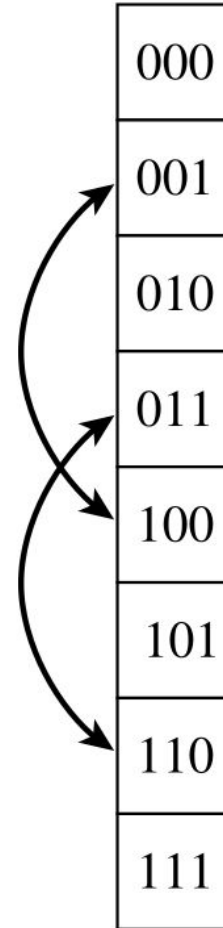


Fast Fourier Transform

Combination – So, we take the original vector of data f_j and rearrange it into bit-reversed order, so that the individual numbers are in the order not of j , but of the number obtained by bit-reversing j .

Then the application of the recursive Danielson-Lanczos Lemma becomes simple. The points as given are the one-point transforms. We combine adjacent pairs to get two-point transforms, then combine adjacent pairs of pairs to get 4-point transforms, and so on, until the first and second halves of the whole data set are combined into the final transform.

Bit-reversal and *combination* are the 2 steps of the algorithms that implement this procedure!



Python built-in functions

Fourier Transforms (`scipy.fft`)

Contents

- Fourier Transforms (`scipy.fft`)
 - Fast Fourier transforms
 - 1-D discrete Fourier transforms
 - 2- and N-D discrete Fourier transforms
 - Discrete Cosine Transforms
 - Type I DCT
 - Type II DCT
 - Type III DCT
 - Type IV DCT
 - DCT and IDCT
 - Example
 - Discrete Sine Transforms
 - Type I DST
 - Type II DST
 - Type III DST
 - Type IV DST
 - DST and IDST
 - Fast Hankel Transform
 - References

<https://docs.scipy.org/doc/scipy/tutorial/fft.html>

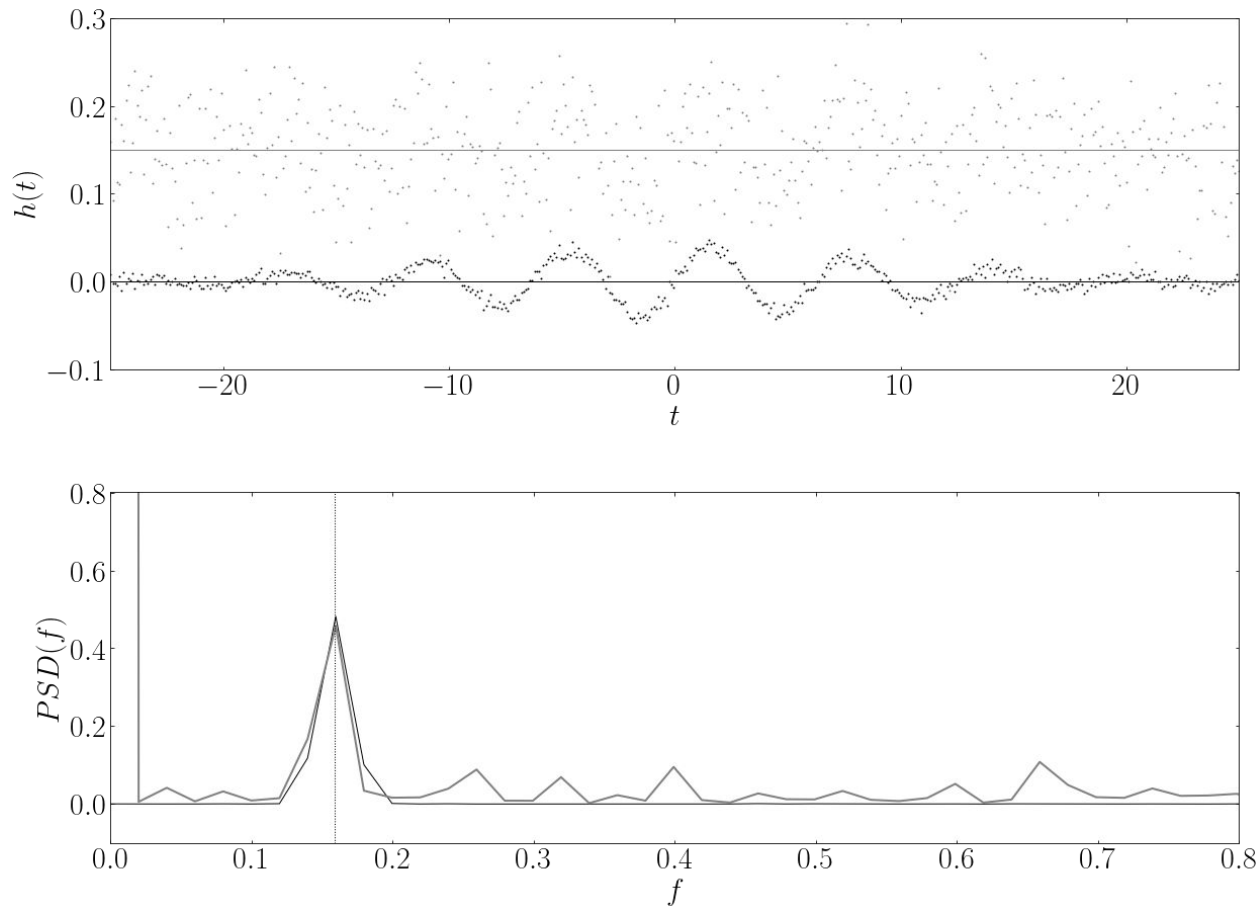
<https://docs.scipy.org/doc/scipy/reference/fft.html#module-scipy.fft>

Python built-in functions

Example

using astroML package

https://www.astroml.org/book_figures/chapter10/fig_fft_example.html



Python built-in functions

Example

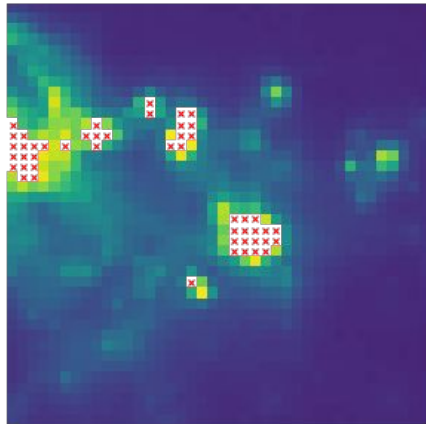
convolution and filtering
in astropy using FFT

<https://docs.astropy.org/en/stable/convolution/>

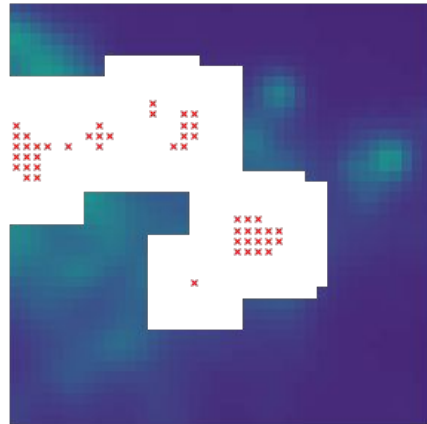
`astropy.convolve_fft`

https://docs.astropy.org/en/stable/api/astropy.convolution.convolve_fft.html#astropy.convolution.convolve_fft

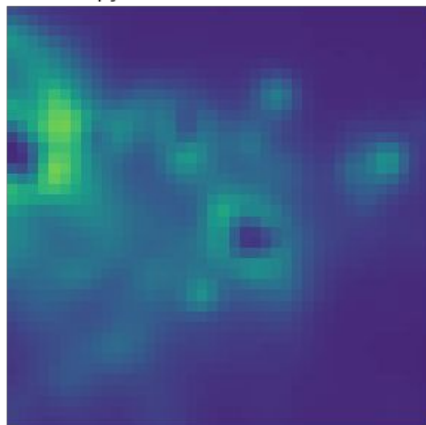
Input Data



Scipy convolved



Scipy convolved (NaN to zero)



Astropy convolved

